

Secure Web Based Password Manager -

Lorenz Ritsch

Introduction

While developing a secure web-based password manager, I had to consider common Web Application Security vulnerabilities such as SQL Injection, XSS, CSRF and aspects like Authentication and Access control. Additionally, because this is a password manager, implementing strong encryption was essential. The result was a multi-user password manager Web application named **Password123** and is currently available at password123.pw.

Implementation

In this Chapter I will explain how I implemented each component and which security techniques I used.

Technology Stack

Backend

- **Framework:** FastAPI (Python)
- **ORM:** SQLAlchemy
- **Database:** PostgreSQL
- **Migrations:** Alembic
- **Caching / Token Blacklist:** Redis
- **Web Server / Reverse Proxy:** Nginx

Frontend

- **Framework:** Svelte (Vite build system)
- **Language:** JavaScript
- **Static Deployment:** Nginx serving compiled assets

Authentication and Access Control

I implemented JWT tokens to authenticate and authorize the user. All API endpoints (except login) require the JWT token included in the “access_token” cookie. The JWT access token cookie is generated server-side and signed with a secret key using HMAC SHA-256 algorithm.

After successful authentication, the server sends a JWT access token cookie in the response. The JWT access token cookie is set to be “httponly=True”, meaning it will only be sent along http request and cannot be accessed by JavaScript and is therefore immune against potential XSS.

The JWT token once generated is only valid for 30 minutes before it will expire and the user must log in again.

Decoded JWT access token payload breakdown:

```
{  
  
  "sub": "c0e1988c-8fdb-4adb-ac1d-079ff8b196bd",  
  
  "exp": 1763241470,  
  
  "jti": "aa2b54ee-6892-41ff-82c4-5a2b656a6fea"  
}
```

The first key “**sub**” stands for Subject, and it contains the user id (UUID) and it is used to authenticate the user, so on every request the JWT is decoded, and signature is verified to identify the user making the request to for authentication and to check if he owns the requested resources.

The second key “**exp**” stands for expire and contains a Unix-timestamp when the token will expire.

Lastly, “**jti**” is a unique identifier and I use it to prevent the JWT from being used after the user logs out. In my application there is a logout button, and it will put the current JWT on a blacklist by using a Redis cache. It will be removed from the Redis cache after it would

usually expire. Therefore on every use of a JWT token the backend checks if this JWT token is on the blacklist.

Two-Factor Authentication (TOTP)

My password manager provides Two-Factor Authentication; however, it is not mandatory, but the user can enable it in the user's profile panel. The database stores the shared OTP secret in the user's table, which is a randomly generated base32 string acting as the seed for further codes.

When the user tries to log in, the system first verifies the password and then checks if the user has two-factor authentication enabled, if True the system will return a short lived "preauth_token" (JWT) that expires after 2 minutes and is stored in local storage. This preauth token enables the user to only call the 2fa-verify endpoint along with the 2fa code.

Preauth_token JWT breakdown (used for the purpose of Two-Factor Authentication):

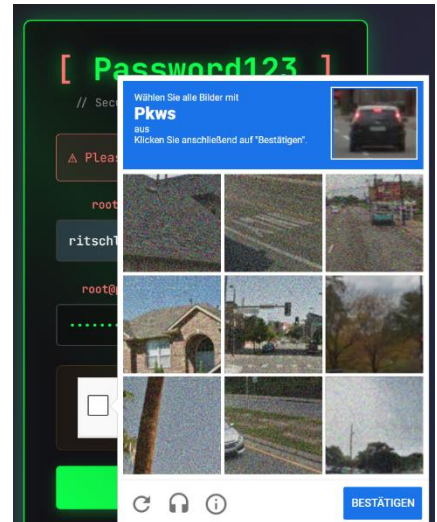
```
{  
  "sub": "c0e1988c-8fdb-4adb-ac1d-079ff8b196bd",  
  "2fa": "pending",  
  "exp": 1763223713,  
  "jti": "895145aa-285b-4b17-8d34-0a19e0fe31de"  
}
```

If the user tries to use this token on another endpoint to bypass 2fa this will not work because it will always check if JWT token has 2fa equals to pending.

```
if payload.get("2fa") == "pending":  
    raise HTTPException(status_code=403, detail="2FA verification required")
```

reCAPTCHA

The login endpoint requires a username, password and a valid reCAPTCHA v3 token and if necessary, also a v2 token. I implemented Google's reCAPTCHA v3 to work in the background while a user logs in, however if the score it gives back is below 0.7 the user must manually complete a reCAPTCHA v2 checkbox before being able to verify his login credentials. This measure mitigates brute-force attacks along with the rate limiting.



```
@router.post("/login", response_model=Union[auth_schema.AuthResponse, auth_schema.PreAuth])
@limiter.limit("5/minute")
async def login(session: PgAsyncSession, credentials: auth_schema.Credentials, request: Request):
    client_ip = request.client.host

    ➔ score = await verify_recaptchav3(token=credentials.recaptcha_token, action="login", remote_ip=client_ip)

    {
        if score < 0.7:
            if not credentials.recaptcha_token_v2:
                raise HTTPException(status_code=403, detail="Please solve reCAPTCHA checkbox")
            if not await verify_recaptchav2(token=credentials.recaptcha_token_v2, remote_ip=client_ip):
                raise HTTPException(status_code=403, detail="Captcha error")
    }
```

Rate Limiting

Endpoints like login, register, change password/email, 2fa verification etc. that could be abused are rate limited to only allow a few calls per minute. Additionally, all other endpoints have a higher rate limit that still mitigates abuse. To implement rate limiting I used the [slowapi](#) library for FastAPI. The following code snippet shows how I have rate limited the login endpoint for 5 requests per minute per client IP, which is another measurement against brute force or similar attacks.

```
@router.post("/login", response_model=Union[auth_schema.AuthResponse, auth_schema.PreAuth])
@limiter.limit("5/minute") ➔
async def login(session: PgAsyncSession, credentials: auth_schema.Credentials, request: Request):
    client_ip = request.client.host
```

If exceeding the limit, the server responds with a http status code: 429 "Too many requests".

Master password:

On registration my application enforces a minimum password length of 10. Additionally, it uses the free [haveibeenpwned API](#) to check if the password is found in any data breach. The API works like this that you must give it the first 5 hash characters of the password, and it will respond with the suffix of the hashes that match that 5-character prefix. This ensures anonymity as there is no risk in leaking the password. The API consumer can then search the results of the response for the presence of their source hash to determine if it really exists in the data set. If it is found it will not allow the user to use it as his master password.

Before storing the password into the database the server hashes it with bcrypt, which I implemented using the passlib library. Bcrypt is ideal for password hashing as it is computationally expensive and therefore makes it difficult to brute force.

End-to-end encryption

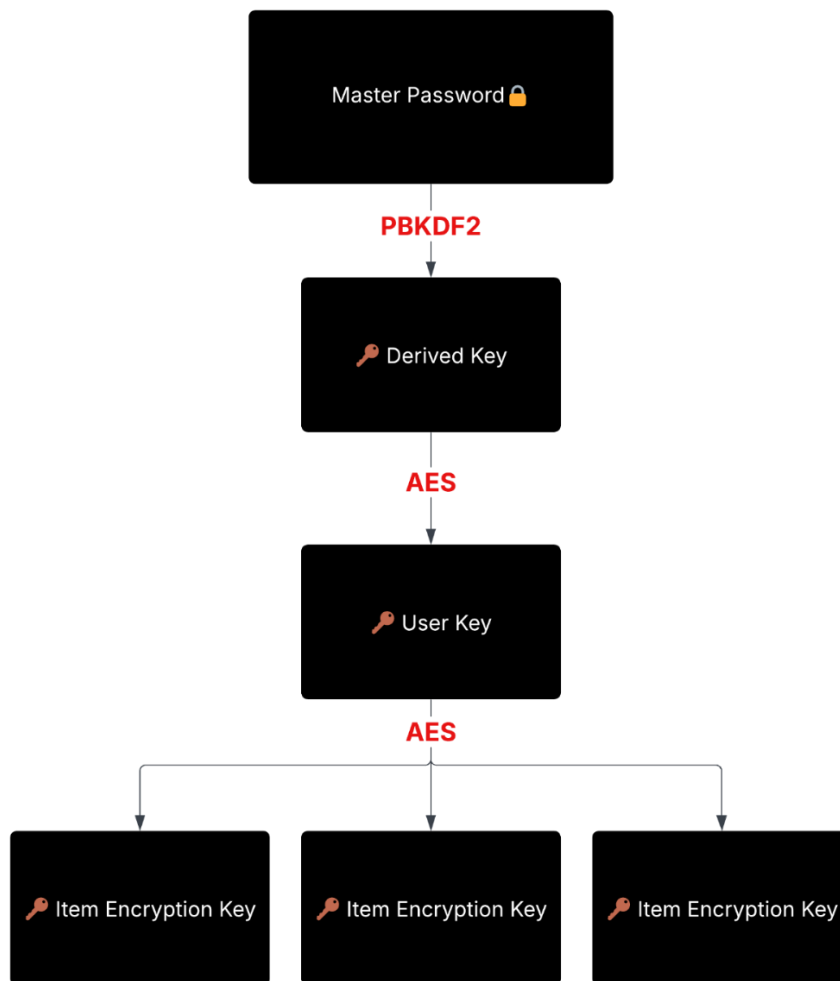
My password manager implements end-to-end encryption to ensure that sensitive credentials remain secure and inaccessible to the server in plaintext form. All encryption and decryption operations occur locally on the client side, preventing any exposure of unencrypted data during transmission or storage. The server acts only as storage with zero knowledge about the content of the encrypted data.

When a user logs in, their master password is locally processed using PBKDF2 (Password-Based Key Derivation Function 2) with a high iteration count (750,000) and a salt to derive a symmetric encryption key. This derived key decrypts the user's master encryption key, which I refer to as the "User Key" and it is stored encrypted in the database along with the salt and initialization vector (both randomly generated during registration). Once decrypted, the User Key is used locally to unlock the individual secret keys for each stored item (credentials, notes, or credit cards), allowing the plaintext data to be viewed in the browser.

This layered approach offers several advantages. If the user changes their master password, only the User Key needs to be re-encrypted with the new derived key, therefore avoiding the need to re-encrypt all individual secrets. Additionally, assigning a unique secret key to each item enables potential future features like secret sharing, where only the secret key (not the entire User Key) would need to be shared with others.

When successfully logged in, the encrypted User Key is stored in a Svelte store in memory only and never persisted anywhere or sent to the server, ensuring it is cleared automatically

when the user logs out, the page is closed or even if you just reload the page it's gone and you are redirected to login.

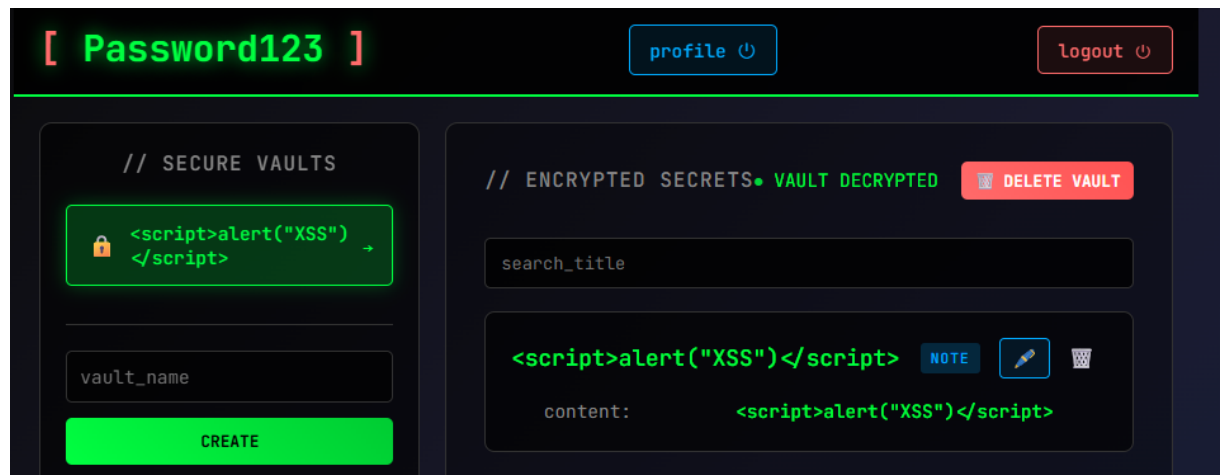


Encryption during transit (HTTPS)

All data is secured during transmission to the server by using HTTPS (Let's Encrypt certificate). Additionally, I configured Nginx to send the HTTP Strict Transport Security header to force the user to always use https.

XSS

The frontend framework Svelte automatically escapes all dynamic values in templates, preventing script injection by default. I never used `{@html}` or any other method that inserts raw HTML into the DOM. All user generated or external data is treated as plain text, not markup. In the following image you see I conducted basic XSS testing but as expected it would not render.



Furthermore, my Content Security Policy header ensures that only necessary scripts like reCAPTCHA are allowed to be loaded by the browser to decrease the risk of XSS.

CSRF Protection

To protect against CSRF, I implemented a Signed Double Submit Cookie as described in the [CSRF Prevention Cheat Sheet](#) by OWASP. On successful login the server sets the “csrf_token” cookie, with SameSite set to “strict”. On every state changing request like POST, PATCH etc. the client sends this cookie in 2 ways to the server:

1. As a cookie
2. In the HTTP Request Header “X-CSRF-Token”

The backend compares the CSRF token from the Cookie with the CSRF token from the header and if they match and the signature verification succeeds the request is legitimate. The CSRF token itself is a random value signed with a secret key through HMAC SHA-256, this prevents manipulating the cookie on client side and is necessary to prevent attacks like Cookie Injection.

Alternatively, I could have also chosen a synchronizer token pattern that would generate the token per session or even per-request. However, my used method is stateless and therefore it was easier to implement with my stateless JWT authentication.

SQL Injection

I am using SQL Alchemy as my ORM for convenience and for security which prevents SQL Injection, because it uses parameterised queries by default.

The following code snippet shows how I handle queries on server-side with SQLAlchemy:

```
@router.get("/vault")
async def get_vaults(session: PgAsyncSession, current_user: user_model.User = Depends(get_current_user)):
    # parameterized query
    stmt = select(vault_model.Vault).where(vault_model.Vault.owner_id == current_user.id)
    result = await session.execute(stmt)
    vaults = result.scalars().all()

    return vaults
```

Clickjacking

To prevent Clickjacking, I simply added the X-Frame-Options “DENY” Header on every response. For this I had to add the following line to my Webserver configuration /etc/nginx/sites-available/default:

```
add_header X-Frame-Options "DENY";
```

Error handling

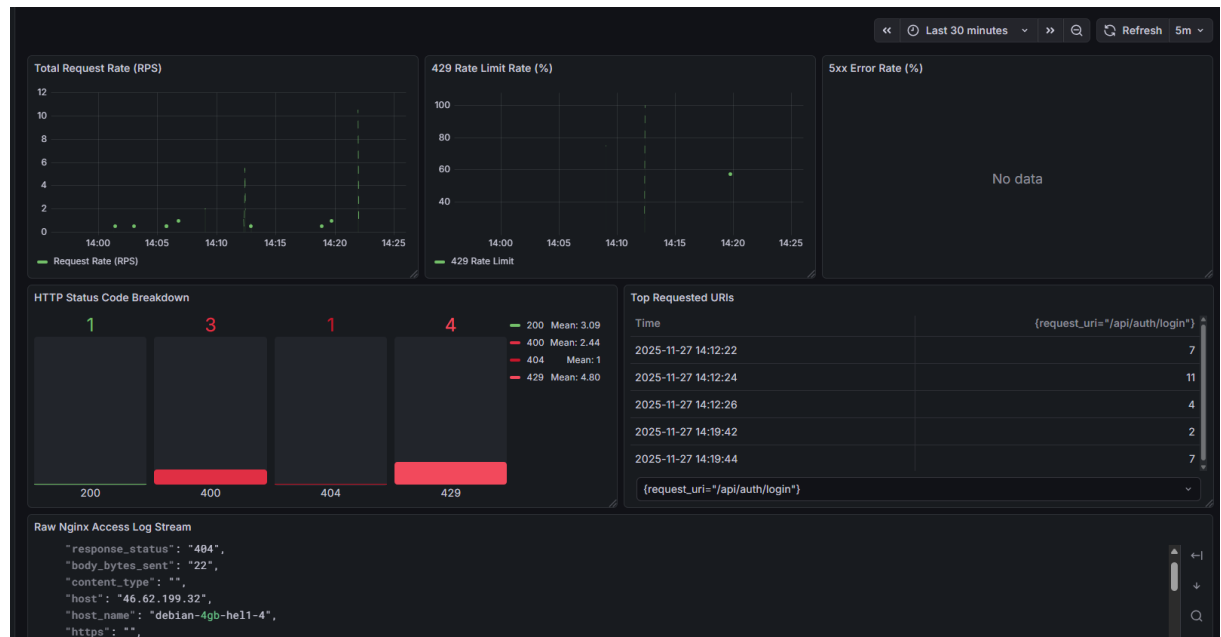
User-facing errors are generic and do not leak stack traces or internals. For example, the most important one being login endpoint returning “Login failed” regardless of wrong username or wrong password.

Logging and Monitoring

As I learned in Logging and Monitoring course, I implemented an observability stack. Nginx writes Json parsed logs into /var/log/nginx/access.json, which are then scraped by Promtail, pushed to Loki and visualized in Grafana. For security reasons I have Grafana container only exposed on localhost:3000 and I am accessing it through SSH Port

Forwarding

locally.



This dashboard is far from being perfect, but I see rate limit hits, 5xx errors, request rate, top requested URIs, HTTP status codes and at the bottom panel the raw Nginx logs.

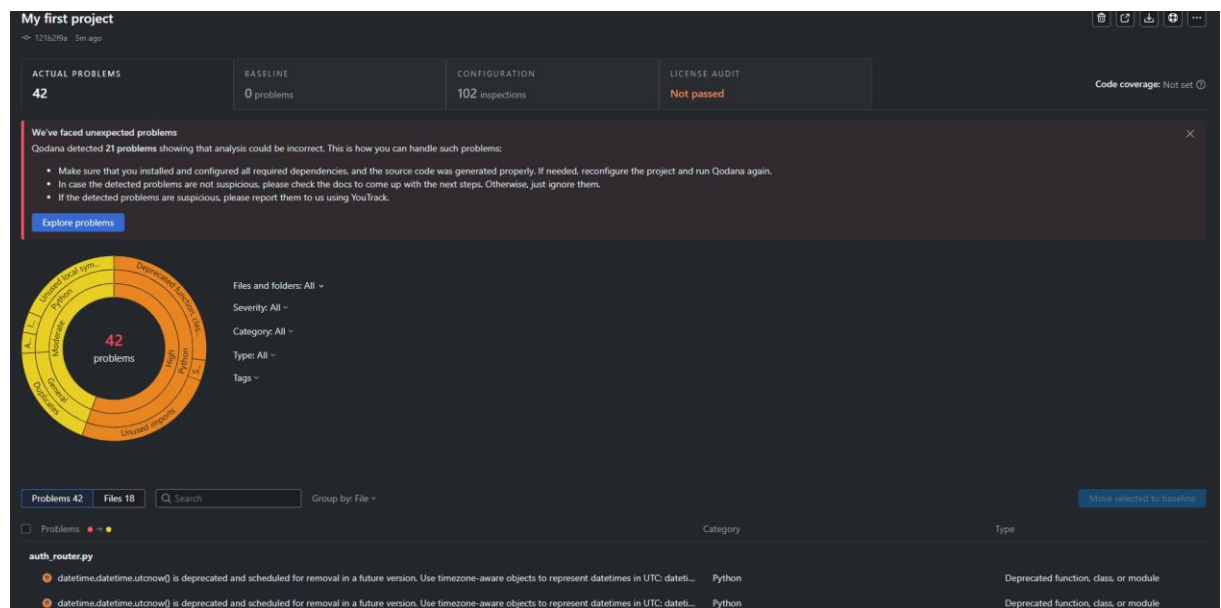
Security Analysis

Static Analysis

In this chapter I will briefly discuss my static analysis experience using JetBrains Quodana and SonarQube cloud.

JetBrains Quodana

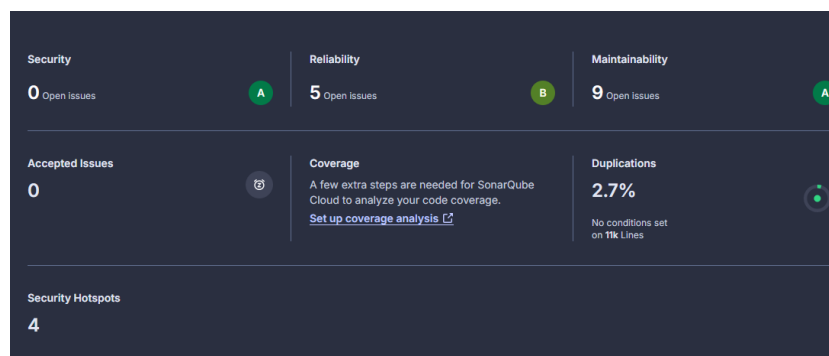
First, I tried JetBrains Quodana and on the first run it detected 42 problems.



Most of the problems were unused imports, which I removed. In the end, I managed to get my code from 42 problems to 28. However, I reviewed all 28 problems and concluded that most are false positives.

SonarQube

Afterwards I also tried SonarQube cloud and fixed some bad practices like code duplication, and I also reviewed the 4 Security Hotspots it found and concluded that they were secure.

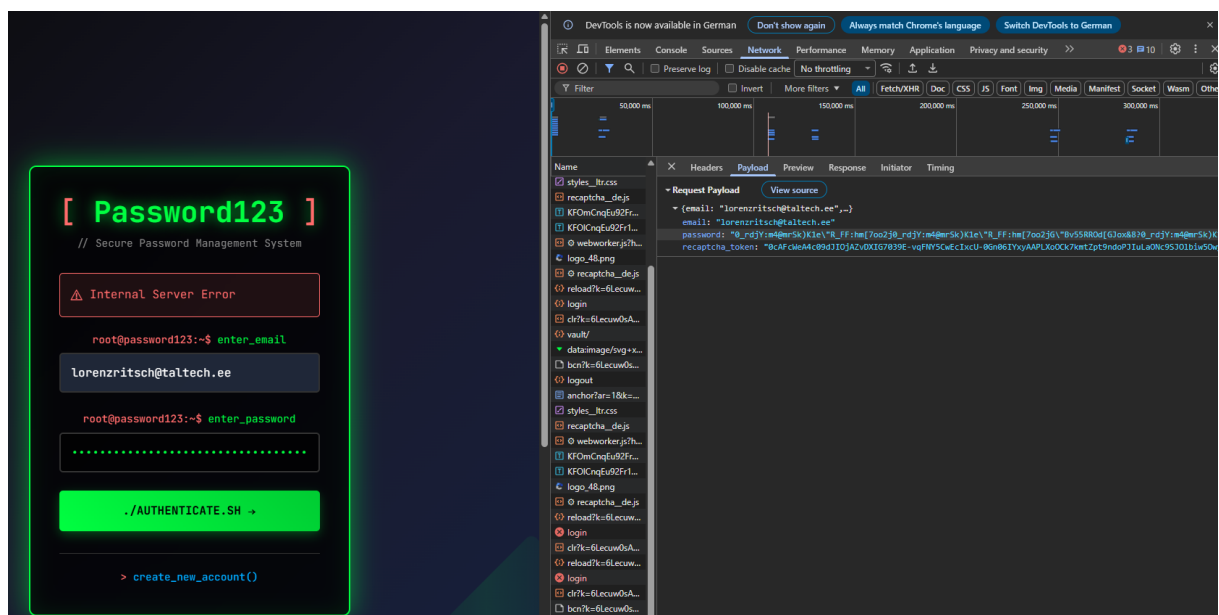


Dynamic Testing

In this chapter I describe any additional dynamic testing I conducted after thinking I have finished programming, so most testing of each component was already done during development. I already tried to describe why it is safe in the Implementation chapter and also tested each component, therefore in this chapter I show selected examples.

1. Triggering Internal Server Error through sending large input

I triggered an error when sending a password that is extremely long, which internally triggered a 500 Internal Server Error.



This made me realise I had to enforce stronger input validation, so on every frontend input field I added the “maxlength” attribute to limit the maximum allowed length of a string inputted by the user. Moreover, on the backend I also added max length limits to ensure if someone bypasses the client-side validation it will still check it on server-side and not throw a 500 error to the client but a “422 Unprocessable Content”. Every request gets validated by Pydantic Python library, which allows me to define what each request body should look like. Furthermore, I set the nginx client_max_body_size to 10 kilobytes, which ensures that no one sends a large payload with malicious intent.

2. Critical bug in rate limiting implementation

For this test I used 2 different devices with different IP addresses to check if my rate limiting even works per client IP and not block all requests if someone hits the limit. I found out that if someone hits the limit it will block all requests from everyone else. The issue was because my API application is running behind a reverse proxy (Nginx) and the rate limiter grouped all traffic under that single proxy IP. To fix this I had to configure the limiter to use the “X-Forwarded-For” header for the original client IP address. This bug showed me that you should not just assume something works but always test if it works as intended.

3. What happens if haveibeenpwned API service is down?

Another availability test, I found out my code didn't have any fallback if the external haveibeenpwned API service is down or unavailable. My application would just throw an error when checking the password for breaches and it would not work to register or to add any credentials. However, after identifying the issue I quickly fixed it by implementing exception handling for this case.

4. Is my JWT token authentication safe?

I also played around with my JWT tokens: I tried to alter the payload, used expired tokens or tokens that are blacklisted because a user logged out already. My system passed all these tests, and I made sure that my authentication is bulletproof.

To summarize the Security Analysis chapter, I can say that I have reduced problems ranging from code duplication or unused imports to more serious vulnerabilities or bugs. I found both static and dynamic analysis useful as it helped me to find problems. However, testing your own code is not sufficient in the real world, because you are biased and other people think from another perspective as I found myself sometimes already assuming it would work.

Conclusion

In conclusion I can say that I have learned many new things by applying my knowledge gained from the lectures into my project. I also encountered some challenges along the way, like for example how to implement encryption had to be well thought out and it took me a couple of days to understand and then implement something secure. I researched how other password managers like Bitwarden handle encryption and I came up with something simpler but still secure enough for my project. Doing this made me really understand zero knowledge encryption with its benefits and disadvantages. I am very grateful for this project and all the lectures as I really learned a lot about Web Application Security.

Sources

https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

<https://haveibeenpwned.com/API/v3>

<https://slowapi.readthedocs.io/en/latest/>

<https://www.jwt.io/>

<https://developers.google.com/recaptcha?hl=de>

<https://en.wikipedia.org/wiki/PBKDF2>